

TrashUQ

Edge AI + Federated Learning for Real-Time Trash Classification
Monitoring

Edge-to-Cloud Monitoring Platform with Deployed Validation

Formal Project Presentation
Advanced Engineering & Federated Systems

Motivation

URBAN WASTE CHALLENGES

Traditional waste management lacks real-time visibility into classification accuracy and node health at the edge.

EDGE INTELLIGENCE

Processing data at the source (Arduino-class nodes) reduces latency and bandwidth usage compared to cloud-only processing.

COLLABORATIVE LEARNING

Federated Learning (FL) allows nodes to improve global models without sharing raw, sensitive data.

OPERATIONAL VISIBILITY

A centralized dashboard provides real-time telemetry and monitoring of the entire edge-to-cloud pipeline.

Project Goals

- **Edge Classification:** Real-time material identification on constrained hardware.
- **Live Telemetry:** Seamless ingestion of metrics and events via MQTT.
- **Persistent Backend:** Robust storage for historical analysis and traceability.
- **Real-Time Dashboard:** Centralized observability of fleet status and FL progress.
- **Federated Coordination:** Managed global model updates with concurrency handling.
- **Multi-Client Validation:** Rigorous testing on physical Arduino-class hardware.

System Architecture

—

Unified edge-to-cloud pipeline integrating telemetry, persistence, and federated learning.

Edge Layer

The system utilizes **Arduino-class devices** as primary edge nodes.

- **Hardware:** Arduino UNO Q MPU series.
- **Telemetry:** MQTT-based event and metric streaming.
- **Inference:** TFLite-based material classification path.
- **Validation:** Controlled synthetic frames for isolating pipeline paths.

DEVICE INVENTORY

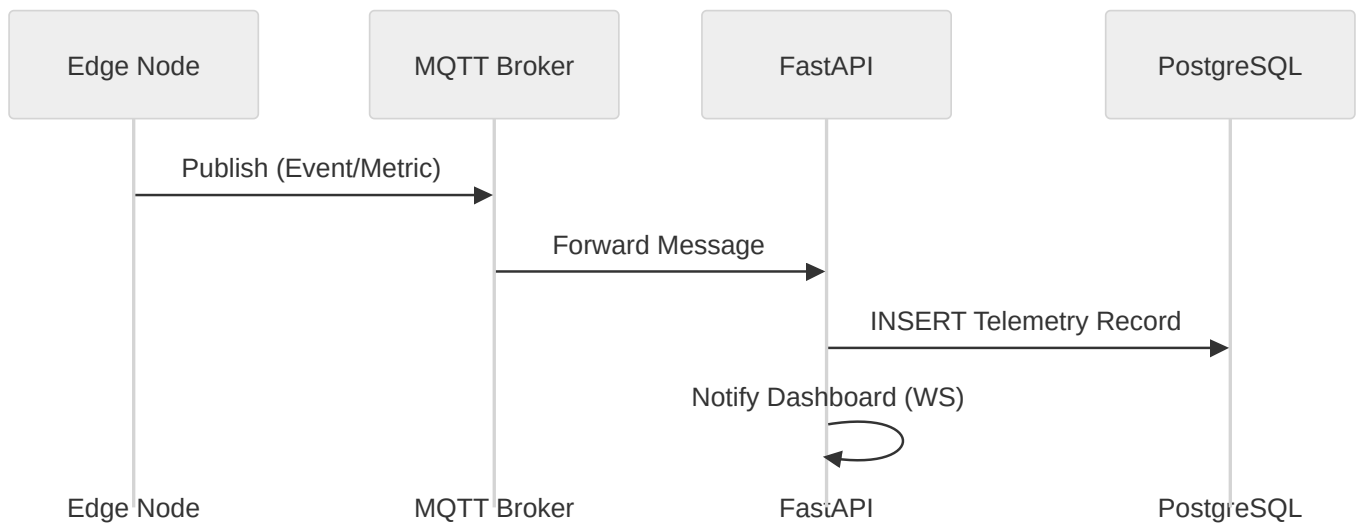
Node ID	Hardware	IP Address
unoq-01	UNO Q MPU	172.20.10.7
unoq-02	UNO Q MPU	172.20.10.2

MQTT + Backend Pipeline

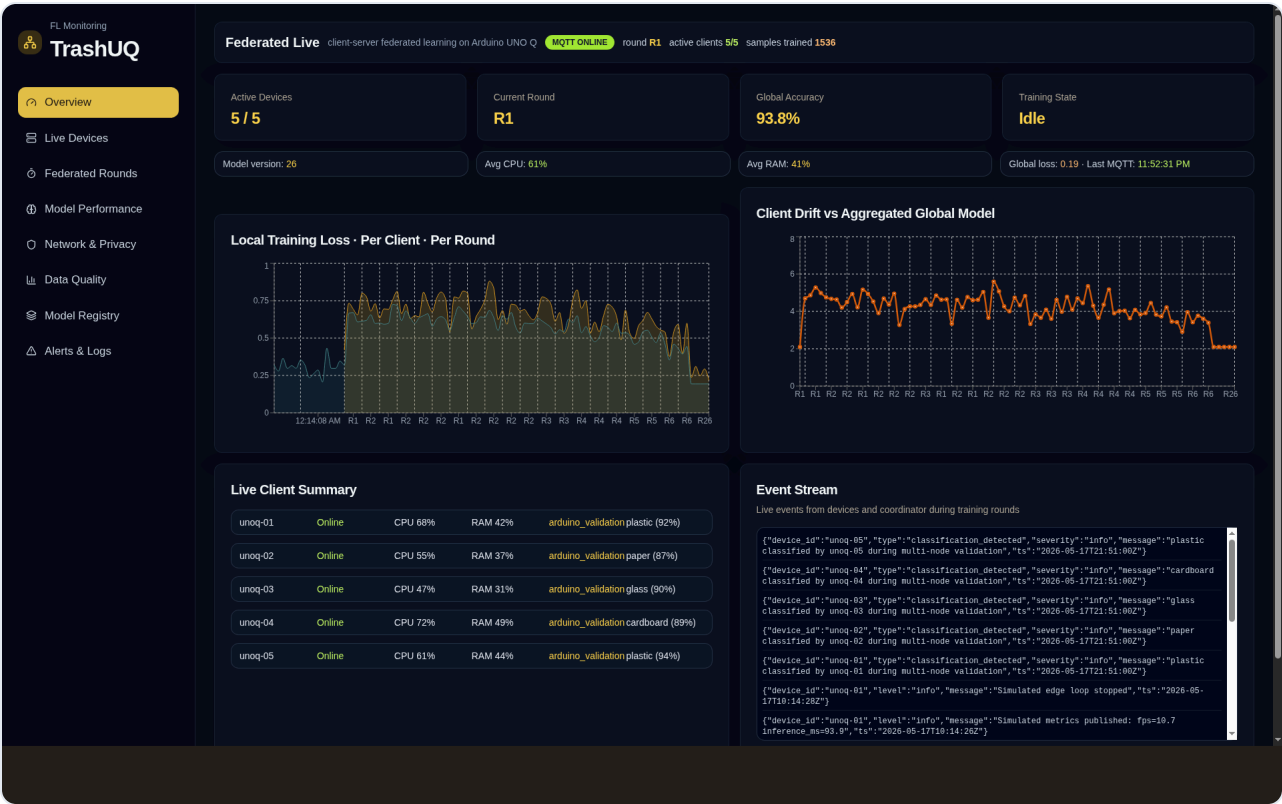
TOPIC STRUCTURE

```
arduino/<device-id>/status  
arduino/<device-id>/metrics  
arduino/<device-id>/classification  
arduino/<device-id>/event  
arduino/<device-id>/logs
```

- Backend subscribes to `arduino/+/#`.
- Stateless MQTT handlers for high throughput.
- PostgreSQL persistence for all telemetry.



Real-Time Dashboard



Observability: Real-time heartbeat and resource monitoring (CPU/RAM).

Analytics: Live event streams and FL progress visualization.

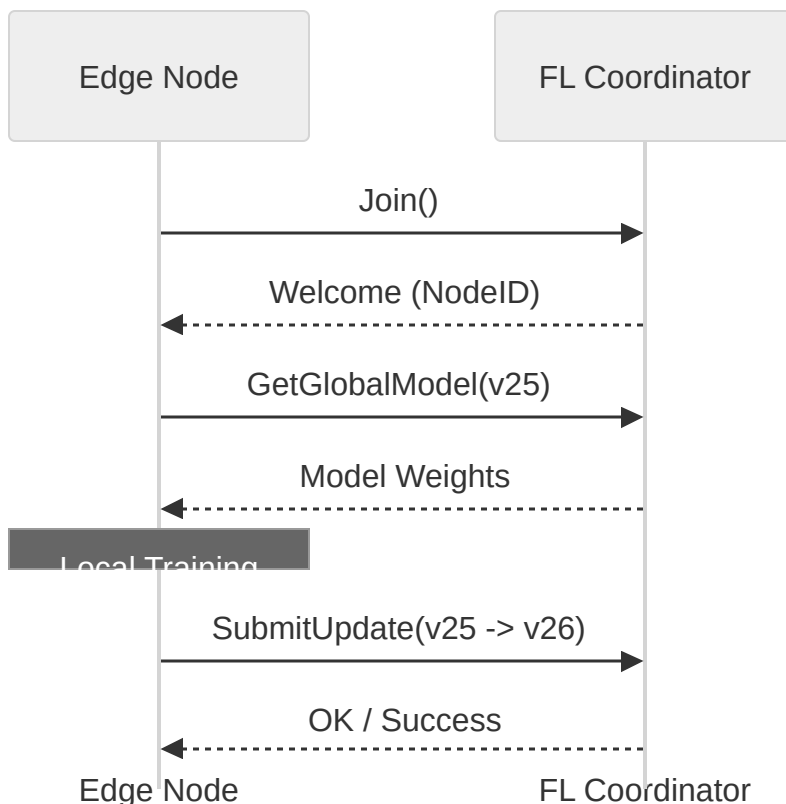
Federated Learning Coordination

GRPC PROTOCOL

- `Join`: Registration of new edge nodes.
- `GetGlobalModel`: Fetching current model head.
- `SubmitUpdate`: Pushing local gradients/weights.

ALGORITHM

- **FedAvg**: Federated Averaging.
- Monotonic model versioning ($0 \rightarrow N$).
- Synchronous aggregation rounds.



Part A: Deployed Validation

2

EDGE NODES

282

MQTT MESSAGES

26

FL ROUNDS

25

AGGREGATIONS

83.3%

SUCCESS RATE

0%

INVALID PAYLOADS

"Controlled synthetic frames isolated the telemetry, persistence, and FL coordination path."

Part A: Node-Level Results

Metric	unoq-01	unoq-02
Total Messages	147	135
Event Msgs	104	100
Mean Accuracy	0.921	0.871
Mean Loss	0.236	0.311
Activity Window	12m 42s	10m 02s

Dashboard Update Lag: < 1s | Median FL Latency: ~25ms

Concurrency & Version Control

In a distributed system, parallel submissions can lead to race conditions. TrashUQ implements **Monotonic Versioning** to protect model consistency.

THE CHALLENGE

Nodes may train on v25 while the coordinator has already progressed to v26.

THE SOLUTION

Stale updates are detected and correctly rejected. During validation, **5 stale parallel updates** were successfully handled.

Robust Concurrency Handling

Data Integrity

Version Consistency

Part B: FL Scalability Study

- **Setup:** FedAvg with 2, 5, 10, 20 clients.
- **Data:** non-IID Dirichlet distribution ($\alpha=0.3$).
- **Duration:** 25 rounds, 3 seeds per setting.
- **Performance:** Consistent 93–94% accuracy.

Results Summary

SYSTEM RELIABILITY

- 0% Invalid Payloads
- 0% Dropped Messages
- < 1s Dashboard Update Lag

FL PERFORMANCE

- 93-94% Final Accuracy
- 25 Successful Aggregations
- 25ms Median Submit Latency

282

TOTAL VERIFIED MESSAGES

System Demonstration Flow



```
# Infrastructure Setup
docker compose up --build
```

```
# Edge Execution
python3 edge/main.py --node unoq-01
```

Lessons Learned

- **MQTT Scalability:** Highly effective for constrained edge node telemetry.
- **Persistence:** Centralized SQL state is essential for auditability.
- **Observability:** Real-time dashboards significantly improve debugging efficiency.
- **Version Control:** Monotonic versioning is a prerequisite for robust FL coordination.
- **Controlled Validation:** Synthetic frame testing ensures pipeline stability before field deployment.

Conclusion

TrashUQ demonstrates a **validated Edge AI + Federated Learning platform** capable of real-time monitoring and collaborative model improvement.

- End-to-end integration from Arduino-class hardware to cloud dashboards.
- Verified robustness against message loss and concurrency races.
- Scalability evidence supporting multi-node federated deployments.

PROJECT READY FOR PRODUCTION DEPLOYMENT

Thank You

Questions?

TrashUQ Project | 2026
docs.trashuq.io

Speaker notes